

1. Persiapan Awal (API Key)

Sebelum coding, langkah pertama adalah mendapatkan akses ke Google Gemini API.

Langkah-langkah:

1. Buka [Google AI Studio](https://aistudio.google.com/).
2. Login dengan akun Google.
3. Klik **"Get API key"**.
4. Klik **"Create API key"** (bisa pilih project baru atau yang sudah ada).
5. Copy key yang muncul (dimulai dengan `Alza...`).

2. Konfigurasi Environment

Agar aman, API Key tidak boleh ditulis langsung di code (hardcoded). Kita menyimpannya di file ``.env``.

File: ``.env``

```
GEMINI_API_KEY=AIzaSyDxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

File: `config/services.php` (Optional, tapi good practice)

Kami mengaksesnya langsung via `env()` di controller demi kesederhanaan, tapi idealnya bisa via config.

3. Implementasi Backend (Laravel)

Seluruh logika AI terpusat di `app/Http/Controllers/AiController.php`.

A. Helper Function (Wrapper)

Kami membuat fungsi private `callGemini` untuk menangani HTTP Request ke Google, supaya tidak perlu menulis ulang kode request berkali-kali.

```
private function callGemini(string $prompt, ?string $imageBase64 = null, ?string $mimeType = null)
{
    $apiKey = env('GEMINI_API_KEY');
```

```

// Setup payload (Text-only atau Multimodal/Gambar)
if ($imageBase64 && $mimeType) {
    $contents = [[
        'parts' => [
            ['inline_data' => ['mime_type' => $mimeType, 'data' =>
$imageBase64]],
            ['text' => $prompt]
        ]
    ]];
} else {
    $contents = [[
        'parts' => [['text' => $prompt]]
    ]];
}

// HTTP Request ke Gemini 2.5 Flash
$response = Http::timeout(60)->post(
    "https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-
flash:generateContent?key={$apiKey}",
    [
        'contents' => $contents,
        'generationConfig' => [
            'temperature' => 0.7, // Kreativitas sedang
            'maxOutputTokens' => 4096,
        ]
    ]
);

return $response->json()['candidates'][0]['content']['parts'][0]['text'] ?? null;
}

```

B. Fitur 1: Smart Dashboard Insight

Memberikan analisis singkat di dashboard. Prompt dibuat sangat ketat agar outputnya konsisten.

```

// Prompt Construction
$prompt = "Berikan 3 poin insight SANGAT SINGKAT...
Data:
- Pemasukan: Rp{$totalIncome}
- Pengeluaran: Rp{$totalExpense}
...
Format Output WAJIB (Gunakan Emoji):
⚠️ [Peringatan]
💡 [Saran]
📊 [Status]";

// Call AI
$insight = $this->callGemini($prompt);

```

C. Fitur 2: Chatbot Keuangan (Context Aware)

Chatbot ini "tahu" kondisi keuangan user karena kita menyuntikkan data (Context Injection) ke dalam prompt sebelum pertanyaan user.

```
public function chat(Request $request) {
    // 1. Ambil data keuangan dari Database
    $totalIncome = Transaction::where(...)->sum('amount');
    $topCategory = ...;

    // 2. Susun System Prompt
    $systemPrompt = "Kamu adalah 'G-ment', asisten keuangan...
        Data User:
        - Saldo: Rp{$balance}
        - Kategori Terboros: {$topCategory}
        ...";

    // 3. Gabung dengan pertanyaan user
    $finalPrompt = $systemPrompt . "\nUser bertanya: " . $request->message;

    // 4. Kirim ke AI
    $reply = $this->callGemini($finalPrompt);

    return response()->json(['reply' => $reply]);
}
```

D. Fitur 3: Scan Receipt (Vision AI)

Menggunakan kemampuan Gemini membaca gambar untuk ekstrak data struk menjadi JSON.

```
public function scanReceipt(Request $request) {
    // 1. Encode gambar ke Base64
    $image = $request->file('receipt_image');
    $base64 = base64_encode(file_get_contents($image));
    $mime = $image->getMimeType();

    // 2. Prompt khusus JSON Extraction
    $prompt = "Ekstrak info struk ini ke format JSON:
        {merchant_name, date, total_amount, items...}";

    // 3. Panggil AI dengan Gambar
    $jsonResult = $this->callGemini($prompt, $base64, $mime);

    // 4. Decode JSON & Auto-Categorize (Logic tambahan di sistem)
    $data = json_decode($jsonResult);
    // ... logic pencocokan kategori ...
}
```

4. Integrasi Frontend (JavaScript)

Di sisi tampilan (Blade), kita menggunakan AJAX/Fetch untuk memanggil API backend ini agar halaman tidak perlu refresh (loading state).

Contoh: Memanggil Insight di Dashboard

```
<div x-data="{ insight: 'Loading...', loading: true }" x-init="
  fetch('{{ route('ai.insight') }}')
  .then(res => res.json())
  .then(data => {
    insight = data.insight;
    loading = false;
  })
">
  <p x-text="insight"></p>
</div>
```

Alur Data Frontend:

1. User buka Dashboard.
2. JS secara background request ke `/ai/insight`.
3. Laravel proses data -> kirim ke Gemini -> dapat teks.
4. Laravel balikkan JSON ke JS.
5. Teks muncul di layar.

5. Ringkasan System

Komponen	Deskripsi
:---	:---
Model	`gemini-2.5-flash` (Cepat & Murah/Gratis)
Library	Built-in Laravel `Http` Client (Tanpa library tambahan berat)

Security	API Key di Server-side, User tidak bisa lihat key.
Context	Data keuangan disuntikkan realtime setiap request chat.